



HASHDIT

HashDit

Smart Contract Security

ListaDao MoolahVault Contract Size Limit Optimization Security Audit Report

ListaDao

30 June 2026 – 3 July 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	30 Jun 2026 – 3 July 2026
Overall Risk	Informational
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/tree/feat/vault-lib-eip170-v2
Commit	feat/vault-lib-eip170-v2

Smart Contract List

No.	Contract Name	Verdict	Details
1	MoolahVault.sol	Informational	[I01]
			[I02]
			[I03]
			[I04]
2	MoolahVaultLib.sol	Informational	[I01]
			[I02]
			[I03]
			[I04]

Findings

[I01] Dead _accruedSupplyBalance() function and unused imports remain in MoolahVault

Contract MoolahVault.sol & MoolahVaultLib.sol

Severity Informational

Description

The MoolahVault._accruedSupplyBalance() function is no longer called anywhere in the contract. Its logic (accruing interest, reading the market state, fetching the vault's supply shares, and converting shares to assets via toAssetsDown) has been moved into MoolahVaultLib, where it is inlined inside both reallocate() and withdrawMoolah().

JavaScript

```
/// @dev Accrues interest on Moolah and returns the vault's assets &
/// corresponding shares supplied on the
/// market defined by `marketParams`, as well as the market's state.
/// @dev Assumes that the inputs `marketParams` and `id` match.
function _accruedSupplyBalance(
    MarketParams memory marketParams,
    Id id
) internal returns (uint256 assets, uint256 shares, Market memory
market) {
    MOOLAH accrueInterest(marketParams);

    market = MOOLAH.market(id);
    shares = MOOLAH.position(id, address(this)).supplyShares;
    assets = shares.toAssetsDown(market.totalSupplyAssets,
market.totalSupplyShares);
}
```

Description

Impact

Because the function is internal and uncalled, the optimizer already excludes it from the deployed bytecode, so its presence does not affect runtime size. It does, however, enlarge the audit surface and leaves several imports dangling.

After removing the function, the following become unused in MoolahVault:

- The SharesMathLib import and its using SharesMathLib for uint256; directive, since toAssetsDown is referenced only inside the dead function.
- The Market type from the IMoolah interface import, since it is referenced only as the dead function's return type.

Independently of the dead function, the following imports are already unused and their presence is not caused by the migration of the _accruedSupplyBalance() logic:

- PendingLib, PendingUint192, and PendingAddress. Note that PendingUint192 and PendingAddress are additionally imported twice (once from the interfaces file and once from PendingLib), and neither the types nor the library are referenced anywhere in the contract.

Separately, the logic now inlined in MoolahVaultLib is duplicated across reallocate() and withdrawMoolah(), which apply the identical accrue then convert with toAssetsDown pattern. This duplication does not exist between the vault and the library, it exists between the two library functions. The related library functions supplyMoolah(), simulateWithdrawMoolah(), and maxDeposit() do not share this exact logic, since supplyMoolah() rounds with toAssetsUp and the view functions use expectedMarketBalances without accruing interest, so they should not be folded into the same helper.

Consider the following:

Recommendation

- Delete the unused _accruedSupplyBalance() function from MoolahVault, then remove the now unused SharesMathLib import together with its using directive and drop Market from the IMoolah interface import.
- Also remove the unused PendingLib, PendingUint192, and PendingAddress imports, including the duplicate imports of PendingUint192 and PendingAddress.

Status

Fixed as recommended, now removes both the unused _accruedSupplyBalance function and the unused imported libraries.

[I02] MoolahVault EIP-170 smart contract size limit considerations

Contract MoolahVault.sol & MoolahVaultLib.sol

Severity **Informational**

Description

Description

The migration successfully brings MoolahVault under the EIP-170 contract size limit. Compiled with the project's own settings (solc 0.8.34, optimizer_runs = 20, via_ir = true), the old vault has a runtime size of 25,774 bytes, which is 1,198 bytes over the

24,576 byte limit and cannot be deployed. The reworked vault measures 23,948 bytes, which is 628 bytes under the limit and deployable, with roughly 1,826 bytes moved out of the vault into MoolahVaultLib. That library has a runtime size of 6,343 bytes and deploys separately, leaving it 18,233 bytes under the limit.

Impact

The concern is that the remaining headroom of 628 bytes is small. Any non trivial future feature added to the vault could push its runtime size back over the 24 KB limit and make it undeployable again, reintroducing the exact problem this change fixes.

The following functions can be considered to be shifted in the future. However, shifting these functions would also entail shifting its corresponding state that is required to be cached in the MoolahVault contract, which can increase complexity.

Recommendation

- updateWithdrawQueue
- _setCap
- The fee math in _accrueFee and _accruedFeeShares.

Status

Acknowledged, ListaDao has indicated that the above functions are noted for future changes if required, given these functions are tightly coupled with vault storage state, so migrating them requires separate refactoring.

[I03] Duplicated supply queue snapshot in _supplyMoolah and _maxDeposit should be folded into MoolahVaultLib

Contract MoolahVault.sol & MoolahVaultLib.sol

Severity Informational

Description

The MoolahVault._supplyMoolah() and MoolahVault._maxDeposit() functions each build the same in memory snapshot of the supply queue before delegating to the library. Both read supplyQueue.length, allocate matching queue and caps arrays, and loop over the queue copying each market id into queue and each config[id].cap into caps. This logic is identical in the two functions, so the same loop is emitted twice inside the vault's bytecode.

Description

JavaScript

```
/// @dev Supplies `assets` to Moolah.  
function _supplyMoolah(uint256 assets) internal {  
    uint256 len = supplyQueue.length;  
    Id[] memory queue = new Id[](len);  
    uint184[] memory caps = new uint184[](len);  
    for (uint256 i; i < len; ++i) {  
        queue[i] = supplyQueue[i];  
        caps[i] = config[supplyQueue[i]].cap;  
    }  
}
```

```

        MoolahVaultLib.supplyMoolah(MOOLAH, queue, caps, assets);
    }

    /// @dev Returns the maximum amount of assets that the vault can supply
    on Moolah.
    function _maxDeposit() internal view returns (uint256) {
        uint256 len = supplyQueue.length;
        Id[] memory queue = new Id[](len);
        uint184[] memory caps = new uint184[](len);
        for (uint256 i; i < len; ++i) {
            queue[i] = supplyQueue[i];
            caps[i] = config[supplyQueue[i]].cap;
        }
        return MoolahVaultLib.maxDeposit(MOOLAH, queue, caps);
    }

```

Impact

This duplication matters because of contract size. The vault is close to the EIP-170 runtime limit as indicated in [IO2].

Note that simply extracting a shared helper does not fix the size aspect unless it is placed correctly. A private helper in the vault, or the MoolahVaultLib library helper that the vault itself calls, is inlined into the vault's bytecode and thus does not reduce contract size, though it does improve readability.

Recommendation

Consider folding the snapshot logic into the library so it physically resides in MoolahVaultLib rather than in the vault, similar to how the `_accruedSupplyBalance` is folded into the MoolahVaultLib for the `reallocate` and `_withdrawMoolah` functions.

Status

Acknowledged, we agree that a fix is complicated given solidity's limitation for storage reference parameters, that is only possible via in-line assembly. This is noted for future references.

[IO4] Documentation inconsistencies

Contract MoolahVault.sol & MoolahVaultLib.sol

Severity **Informational**

Description

The deployment plan for upgrading MoolahVault contains several inaccuracies:

Description

1. The plan describes the upgrade as a Transparent proxy upgrade, but MoolahVault is a UUPS proxy. It inherits UUPSUpgradeable and gates upgrades through `_authorizeUpgrade`, which requires the `DEFAULT_ADMIN_ROLE`. There is no ProxyAdmin contract involved. The upgrade must be performed by calling `upgradeToAndCall(newImpl, data)` directly on the proxy, by the TimeLock

contract, so the accurate description is a UUPS upgrade via `upgradeToAndCall` on the proxy, authorized by `DEFAULT_ADMIN_ROLE` held by the `TimeLock`

2. The plan includes a step to deploy a `VaultConfigLib` library contract that describe extracting the whitelist management logic into a dedicated `VaultConfigLib` library. In the actual implementation, no `VaultConfigLib` exists anywhere in the codebase. Instead, the `setWhiteList` logic was migrated directly into `MoolahVaultLib`, where `MoolahVault.setWhiteList()` delegates to `MoolahVaultLib.setWhiteList()`. The step that deploys `VaultConfigLib` targets a nonexistent contract and should be removed.
3. The plan states the implementation is linked to two libraries, but there is only one link time library. Only `MoolahVaultLib` exposes public functions that compile to a `DELEGATECALL` and therefore require separate deployment and linking, given the `VaultConfigLib` library is not implemented.
4. The plan highlights the upgrade as a single implementation swap via “Upgrade MoolahVault proxy implementation”, but there are multiple `ListaDao` vault deployments. .

Impact

This creates a mismatch between the documented architecture and the deployed code.

Recommendation	Consider updating the deployment documentation to reflect the actual system as mentioned above
-----------------------	--

Status	Fixed as recommended, all points addressed in the new PR description.
---------------	---